Enterprise Financial Research Assistant

Portfolio Case Study

Type	Full-stack AI / MLOps portfolio project
Domain	Financial research · RAG · Knowledge graphs
Stack	Python · FastAPI · LangGraph · Neo4j · Streamlit
Deployment	Docker · Kubernetes · Prometheus / CI

1. Executive Summary

This project demonstrates a production-grade **Retrieval-Augmented Generation (RAG)** platform that allows financial analysts to interrogate large collections of SEC filings, earnings call transcripts, and research PDFs using natural language. The system combines *dense vector search*, *BM25 sparse retrieval*, and a *Neo4j knowledge graph* to deliver cited, confidence-scored answers with full conversation memory — all deployable with a single **docker compose up**.

Key portfolio signals:

- End-to-end AI pipeline: ingestion → retrieval → graph → synthesis → UI
- Hybrid retrieval fusing semantic vectors (all-MiniLM-L6-v2) and BM25 (RRF weighting 65/35)
- LangGraph stateful multi-step workflow with human-in-the-loop approval gate
- FastAPI REST + SSE streaming endpoints with Prometheus observability
- Streamlit analyst UI with file upload, conversation memory, and graph comparison tables

2. Problem Statement

Professional financial analysts spend hours manually reading PDFs to answer questions such as *"Which semiconductor companies had the highest gross margin over the last five years?"*. Generic LLMs hallucinate financial figures and cannot cite primary sources. Enterprise search tools are keyword-only and miss semantic intent.

This project targets three specific gaps:

- **Evidence grounding:** every answer must be traceable to a page/chunk in a source document.
- **Structured data integration:** numerical metrics from filings should feed a queryable graph, not just float in unstructured text.

- **Trust & review:** low-confidence answers must be flagged for human approval before being acted on.

3. System Architecture

The system is composed of four loosely-coupled layers:

Ingestion layer	PyPDF extracts text page-by-page. Pages are chunked with 180-token overlap (chunk size 1 200 chars). SHA-256 deduplication prevents re-indexing. Regex extractors pull gross margin and net margin ratios and persist them to Neo4j.
Retrieval layer	HybridRetriever encodes the query with sentence-transformers, performs a cosine-similarity dense search over a NumPy float32 matrix, then fuses scores with BM25Okapi (0.65 dense + 0.35 sparse). Results persist across restarts via chunks.json + vectors.npy.
Orchestration layer	A three-node LangGraph DAG (retrieve → graph → answer) manages state. The graph node enriches answers when the query mentions financial metrics such as "gross margin". Conversation history (capped at 100 messages / 500 sessions) feeds the answer node for multi-turn coherence.
Presentation layer	FastAPI exposes /api/v1/documents (PDF upload), /api/v1/ask (JSON), and /api/v1/ask/stream (SSE). The Streamlit UI provides file upload, a research question text area, confidence gauge, cited sources table, and graph-comparison dataframe. Prometheus metrics are protected by an optional bearer token.

4. Technical Implementation

4.1 Hybrid Retrieval

Combining dense and sparse signals captures both semantic similarity and exact keyword overlap — critical for financial terminology (e.g. "EBITDA", ticker symbols) where pure vector search can underperform.

```
score = 0.65 × cosine_similarity(qv, dv) + 0.35 × BM25_normalised(q, d)
```

Both score components are normalised to [0, 1] before fusion. Top-*k* is configurable per request (default 8, max 30).

4.2 LangGraph Workflow

LangGraph provides a typed **StateGraph** that guarantees each node receives a validated state dict and propagates only the keys it mutates. The DAG is:

- **retrieve** – runs hybrid search, populates sources
- **graph** – queries Neo4j for metric history when relevant, populates graph_data
- **answer** – calls the OpenAI-compatible LLM with evidence + memory, computes a confidence score, sets status to needs_approval if confidence < 0.55

4.3 Knowledge Graph (Neo4j)

Each ingested filing creates or updates Company and Metric nodes connected by [:REPORTED] edges. Cypher queries aggregate multi-year averages and return ranked comparisons:

```
MATCH (c:Company)-[:REPORTED]->(m:Metric {name:"gross_margin_pct"})
WITH c, m ORDER BY m.year DESC
RETURN c.ticker, reduce(t=0.0, x IN collect(m)[0..5] | t+x.value) / 5 AS avg5yr
```

4.4 Security Controls

- Filename sanitisation (path-traversal prevention) before any disk write
- 50 MB upload cap enforced before any PDF parsing
- CORS origins are configurable (default open for local dev, lock down in prod)
- Prometheus /metrics endpoint protected by configurable bearer token
- No secrets committed; all credentials via .env / environment variables

5. Technology Stack

Web framework	FastAPI 0.118+ with Pydantic v2 models and async handlers
Orchestration	LangGraph 1.x — typed StateGraph, async nodes
Embeddings	sentence-transformers all-MiniLM-L6-v2 (384-dim, float32)
Sparse retrieval	rank-bm25 (BM25Okapi)
LLM backend	OpenAI-compatible endpoint — LM Studio / any OpenAI-API server
Graph database	Neo4j 6+ via official Python driver; bolt:// or neo4j+s://
UI	Streamlit 1.42+ — sidebar upload, metrics, source table, graph view
Observability	Prometheus client — request counters, latency histograms
Containerisation	Docker + docker-compose; Kubernetes manifests in /infra

Testing / CI	pytest; GitHub Actions workflow

6. Key Design Decisions & Trade-offs

Local vector store vs. hosted DB	NumPy + BM25 removes external dependencies for local dev and demos. The README roadmap explicitly targets OpenSearch BM25 + k-NN for production, keeping the code path identical.
LangGraph over plain chains	The StateGraph makes retrieval, graph enrichment, and synthesis individually testable and replaceable without touching downstream nodes.
Sentence-transformer over OpenAI embeddings	Runs fully offline, deterministic, and free. Switch by changing one config key.
In-process memory vs. Redis	Dict-backed memory with LRU eviction (500 sessions, 100 turns each) is sufficient for demo scale; Redis checkpoint integration is pre-wired in config.
Human-in-the-loop threshold	Confidence < 0.55 returns status="needs_approval". Threshold is env-configurable to tune precision/recall trade-off per deployment context.

7. End-to-End Example Workflow

The following illustrates a typical analyst session:

- Analyst uploads *NVDA_2025_10K.pdf* via the sidebar. Ingest returns chunk count, page count, and extracted ratios (e.g. gross_margin_pct: 75.0). The Company/Metric graph is updated.
- Analyst types: *"What drove NVIDIA's gross margin improvement in 2025?"*
- LangGraph **retrieve** node returns top-8 hybrid-scored chunks from the filing.
- LangGraph **graph** node detects "gross margin" → queries Neo4j for NVDA year-over-year metric history.
- LangGraph **answer** node synthesises both evidence streams, cites [n] references, and computes confidence = 0.72 → status = completed.
- The UI renders the answer with inline citations, a confidence badge, a sources dataframe, and a knowledge-graph comparison table.

8. Observability & Operations

Production-readiness signals:

- **Health endpoint:** GET /api/v1/health returns indexed chunk count for liveness/readiness probes.
- **Prometheus metrics:** request count, latency, error rate scraped by any standard collector.
- **Structured logging:** Neo4j write status, memory trim events, and errors are logged via Python's standard logging module for aggregation by CloudWatch / ELK.
- **Docker Compose:** single command spin-up of API + Streamlit + (optional) Neo4j.
- **Kubernetes manifests:** /infra directory contains deployment and service YAML.
- **CI pipeline:** GitHub Actions runs pytest on every push.

9. Production Roadmap

This is an engineering portfolio MVP. The README documents the following production-hardening steps that would be required before institutional use:

- Replace NumPy/BM25 with OpenSearch BM25 + k-NN and reciprocal-rank fusion
- Replace dict-backed memory with Redis-persisted LangGraph checkpoints
- Ingest structured XBRL facts from SEC EDGAR instead of regex ratio extraction
- Add OAuth 2.0 / OIDC, RBAC, document entitlements, and immutable audit logs
- Add encryption at rest and in transit, PII detection, and secrets management (Vault / AWS SM)
- Add source licences for Bloomberg/Reuters data; respect proprietary content terms
- Add evaluation datasets, retrieval quality metrics (MRR, nDCG), and hallucination checks
- Add load tests, canary deployments, and SLO alerting

10. Conclusion

This portfolio project demonstrates the ability to design and ship a complete **AI-powered research platform** from raw data ingestion through to a polished analyst UI — combining modern LLM orchestration (LangGraph), hybrid retrieval (dense + sparse), graph-structured domain knowledge (Neo4j), production API patterns (FastAPI + SSE), and cloud-native operations (Docker, Kubernetes, Prometheus, CI).

The codebase is intentionally minimal (≈ 200 lines of application code) yet production-shaped: each component is independently replaceable, security controls are explicit, and the README roadmap is honest about the gap between portfolio MVP and enterprise deployment.

This is an engineering portfolio project, not an investment-advice product.